Experiment: Application and System Monitoring using Prometheus and Grafana

1. Aim

To implement **application and system monitoring** on Ubuntu using **Prometheus, Node Exporter, and Grafana**, and visualize system performance metrics through a dashboard.

2. Objectives

After completing this experiment, students will be able to:

1. Understand the concept of system monitoring.
2. Install and configure Node Exporter.
3. Install and configure Prometheus.
4. Collect CPU, memory, disk, and network metrics.
5. Query monitoring data using PromQL.
6. Install and configure Grafana.
7. Connect Grafana with Prometheus.
8. Create a system monitoring dashboard.
9. Observe system performance in real time.

3. Software and Hardware Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM recommended
- Internet connection

Software

- Ubuntu 20.04/22.04/24.04 or compatible Ubuntu system
- Terminal
- Web browser

Tools

Tool	Purpose
Node Exporter	Collects Ubuntu system metrics
Prometheus	Collects, stores and queries metrics
Grafana	Visualizes metrics using dashboards

4. Theory

What is System Monitoring?

System monitoring is the process of continuously collecting and analyzing information about a computer system to determine its:

- CPU utilization
- Memory utilization
- Disk usage
- Network traffic
- System availability
- Performance

Monitoring helps administrators detect problems and understand system performance.

5. Role of Each Tool

Node Exporter

Node Exporter runs on the Ubuntu machine and exposes system-level metrics.
It collects information such as:
CPU
Memory
Disk
Network
Filesystem
System load

Prometheus

Prometheus is a monitoring and time-series database.
It:
Collects metrics
↓
Stores metrics
↓
Queries metrics
↓
Provides data to Grafana
Prometheus uses **PromQL** (Prometheus Query Language) to query the collected data.

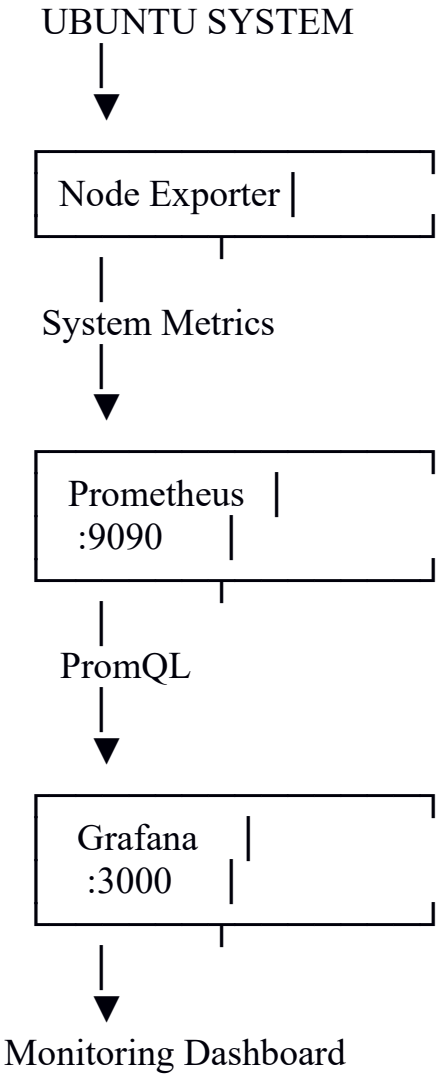
Grafana

Grafana is a visualization and dashboarding tool.
It obtains data from Prometheus and displays it using:

- Graphs
- Gauges
- Tables

- Charts
- Dashboards

6. Architecture



7. Step 1 – Check Ubuntu System

Open Terminal.
uname -m
For the commands in this experiment, the output should normally be:
x86_64
Check Ubuntu version:
lsb_release -a

8. Step 2 – Update Ubuntu
sudo apt update
Then:
sudo apt upgrade -y

9. Step 3 – Install Node Exporter
Node Exporter is used to collect system-level metrics.
Download Node Exporter:
cd /tmp

wget
https://github.com/prometheus/node_exporter/releases/download/v1.12.1/node_exporter-1.12.1.linux-amd64.tar.gz
Extract:
tar -xvf node_exporter-1.12.1.linux-amd64.tar.gz
Copy the executable:
sudo cp node_exporter-1.12.1.linux-amd64/node_exporter /usr/local/bin/
Check the installation:
node_exporter --version

10. Step 4 – Create Node Exporter User
sudo useradd --no-create-home --shell /bin/false node_exporter
Set ownership:
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter

11. Step 5 – Create Node Exporter Service
Create the service file:
sudo nano /etc/systemd/system/node_exporter.service
Enter:
[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple

ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
Save:
CTRL + O
ENTER
CTRL + X

12. Step 6 – Start Node Exporter
Reload systemd:
sudo systemctl daemon-reload
Enable Node Exporter:
sudo systemctl enable node_exporter
Start Node Exporter:
sudo systemctl start node_exporter
Check status:
sudo systemctl status node_exporter
Expected:
Active: active (running)

13. Step 7 – Test Node Exporter
Run:
curl http://localhost:9100/metrics
You should see metrics such as:
node_cpu_seconds_total
node_memory_MemAvailable_bytes
node_memory_MemTotal_bytes
node_filesystem_avail_bytes
node_network_receive_bytes_total
You can also open:
http://localhost:9100/metrics
Observation
Node Exporter exposes Ubuntu system metrics on:
Port: 9100

14. Step 8 – Install Prometheus
Download Prometheus:
cd /tmp

wget
https://github.com/prometheus/prometheus/releases/download/v3.14.0/prometheus-3.14.0.linux-amd64.tar.gz
Extract:
tar -xvf prometheus-3.14.0.linux-amd64.tar.gz
Create directories:
sudo mkdir -p /etc/prometheus
sudo mkdir -p /var/lib/prometheus
Copy Prometheus:
sudo cp prometheus-3.14.0.linux-amd64/prometheus /usr/local/bin/
Copy Promtool:
sudo cp prometheus-3.14.0.linux-amd64/promtool /usr/local/bin/
Copy configuration file:
sudo cp prometheus-3.14.0.linux-amd64/prometheus.yml /etc/prometheus/

15. Step 9 – Create Prometheus User
sudo useradd --no-create-home --shell /bin/false prometheus
Set permissions:
sudo chown prometheus:prometheus /usr/local/bin/prometheus
sudo chown prometheus:prometheus /usr/local/bin/promtool
sudo chown -R prometheus:prometheus /etc/prometheus
sudo chown -R prometheus:prometheus /var/lib/prometheus

16. Step 10 – Configure Prometheus
Open:
sudo nano /etc/prometheus/prometheus.yml
Replace the contents with:

```
global:
  scrape_interval: 15s

scrape_configs:

  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "node_exporter"
    static_configs:
```

- targets: ["localhost:9100"]

Save:
CTRL + O
ENTER
CTRL + X

17. Step 11 – Check Prometheus Configuration

Run:
sudo -u prometheus promtool check config /etc/prometheus/prometheus.yml
Expected:
SUCCESS
This confirms that the Prometheus configuration is valid.

18. Step 12 – Create Prometheus Service

sudo nano /etc/systemd/system/prometheus.service
Enter:

```
[Unit]
Description=Prometheus Monitoring
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple

ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus

Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Save:
CTRL + O
ENTER
CTRL + X

19. Step 13 – Start Prometheus
sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
Check:
sudo systemctl status prometheus
Expected:
Active: active (running)

20. Step 14 – Open Prometheus
Open the browser and enter:
http://localhost:9090
Prometheus web interface should appear.

21. Step 15 – Check Monitoring Targets
In Prometheus:
Status
↓
Targets
You should see:

Endpoint	State
localhost:9090	UP
localhost:9100	UP

Meaning
UP = Prometheus is successfully collecting metrics

DOWN = Prometheus cannot reach the target

22. Step 16 – Test CPU Monitoring
In the Prometheus query box enter:
100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
Click:
Execute
The result represents CPU utilization.

23. Step 17 – Test Memory Monitoring
Enter:

100 * (1 - node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)
This gives memory utilization percentage.

24. Step 18 – Test Disk Monitoring
Enter:
100 * (1 -
node_filesystem_avail_bytes{mountpoint="/",fstype!~"tmpfs|overlay"}
/
node_filesystem_size_bytes{mountpoint="/",fstype!~"tmpfs|overlay"})
This gives approximate root filesystem utilization.

25. Step 19 – Test Network Monitoring
For received network traffic:
rate(node_network_receive_bytes_total[5m])
For transmitted network traffic:
rate(node_network_transmit_bytes_total[5m])

26. Step 20 – Install Grafana
Install required packages:
sudo apt-get install -y adduser libfontconfig1 musl
Download Grafana OSS:
cd /tmp

wget
https://dl.grafana.com/grafana/release/13.2.2/grafana_13.2.2_34846740809_linux_amd64.deb
Install:
sudo dpkg -i grafana_13.2.2_34846740809_linux_amd64.deb
If dependencies are missing:
sudo apt-get install -f -y

27. Step 21 – Start Grafana
sudo systemctl daemon-reload
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
Check:
sudo systemctl status grafana-server
Expected:

Active: active (running)

28. Step 22 – Open Grafana

Open:
http://localhost:3000
Initial login:
Username: admin
Password: admin
You may be asked to change the password.

29. Step 23 – Add Prometheus to Grafana

In Grafana:
Connections
↓
Data Sources
↓
Add data source
↓
Prometheus
Enter the URL:
http://localhost:9090
Click:
Save & Test
The connection should be successful.

30. Step 24 – Create Grafana Dashboard

Go to:
Dashboards
↓
New
↓
New Dashboard
↓
Add Visualization
Select:
Prometheus
Create the following panels.

Panel 1 – CPU Usage

Query:
100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
Panel title:
CPU Usage
Visualization:
Time series

Panel 2 – Memory Usage
Query:
100 * (1 - node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)
Panel title:
Memory Usage

Panel 3 – Disk Usage
Query:
100 * (1 -
node_filesystem_avail_bytes{mountpoint="/",fstype!~"tmpfs|overlay"}
/
node_filesystem_size_bytes{mountpoint="/",fstype!~"tmpfs|overlay"})
Panel title:
Disk Usage

Panel 4 – Network Traffic
Query:
rate(node_network_receive_bytes_total[5m])
Panel title:
Network Traffic
Save the dashboard as:
Ubuntu System Monitoring

32. Step 25 – Test the Monitoring
To create temporary CPU load:
yes > /dev/null &
Observe the CPU graph in Grafana.
Stop the process:
pkill yes

The CPU utilization should return toward its previous level.

33. Useful Commands

Check Node Exporter

sudo systemctl status node_exporter

Restart Node Exporter

sudo systemctl restart node_exporter

Check Prometheus

sudo systemctl status prometheus

Restart Prometheus

sudo systemctl restart prometheus

Check Grafana

sudo systemctl status grafana-server

Restart Grafana

sudo systemctl restart grafana-server

---

34. Important Ports

Tool	Port	URL
Node Exporter	9100	http://localhost:9100/metrics
Prometheus	9090	http://localhost:9090
Grafana	3000	http://localhost:3000

35. Expected Result

The Ubuntu system was successfully monitored using **Node Exporter and Prometheus**, and the collected metrics were visualized using **Grafana**. CPU, memory, disk, and network metrics were successfully displayed on a monitoring dashboard.

36. Conclusion

Prometheus provides the monitoring and time-series data collection layer, while Grafana provides visualization and dashboarding. Node Exporter acts as the bridge between the Ubuntu operating system and Prometheus by exposing system-level metrics.